

Introdução ao JavaScript

Até aqui, o HTML organiza o conteúdo e o CSS cuida da aparência. O JavaScript entra quando queremos que a página **faça alguma coisa**.

No HTML, podemos colocar um `<script>` para escrever o JavaScript:

```
<script>
  alert("Olá, turma!");
</script>
```

Quando o navegador encontra esse código, ele executa o JavaScript e mostra uma mensagem.

1. Primeiro, vamos entender as variáveis

Imagine que você tem várias **caixas com etiquetas**.

Uma caixa pode ter a etiqueta `nome` e guardar `"Maria"`. Outra pode ter a etiqueta `idade` e guardar `17`.

Em JavaScript, uma variável funciona de maneira parecida:

```
let nome = "Maria";
let idade = 17;
```

Podemos ler isso assim:

Crie uma caixa chamada `nome` e coloque `"Maria"` dentro dela.

Crie uma caixa chamada `idade` e coloque `17` dentro dela.

O sinal `=` significa **"receba" ou "guarde"**.

```
let nome = "Maria";
```

Não leia como:

`nome` é igual a `Maria`.

Para começar, é melhor pensar:

`nome` recebe `"Maria"`.

Depois podemos usar o que está guardado na variável:

```
let nome = "Maria";  
  
alert(nome);
```

O navegador mostrará:

Maria

2. Mas o que pode ser colocado dentro da caixa?

A caixa pode guardar diferentes tipos de valores.

String: texto

Strings são textos. Normalmente ficam entre aspas.

```
let nome = "Maria";  
let cidade = "Belo Horizonte";
```

Number: número

Números não precisam de aspas.

```
let idade = 17;  
let altura = 1.75;
```

Compare:

```
let idade = 17;  
let idadeTexto = "17";
```

Apesar de parecerem iguais, um é **Number** e o outro é **String**.

Boolean: verdadeiro ou falso

Um booleano só pode ter dois valores:

```
let aprovado = true;  
let faltou = false;
```

Podemos imaginar o booleano como um **interruptor**:

```
true  → ligado  
false → desligado
```

E o **null** e o **undefined**?

Existem também alguns valores especiais.

undefined significa que uma variável existe, mas ainda não recebeu um valor:

```
let telefone;  
  
console.log(telefone);
```

Resultado:

```
undefined
```

null é usado quando queremos dizer propositalmente:

```
"Aqui não existe nenhum valor."
```

```
let endereco = null;
```

Para começar, concentre-se principalmente nestes três:

```
String  → texto  
Number  → número  
Boolean → verdadeiro ou falso
```

3. JavaScript descobre o tipo sozinho

Em algumas linguagens, precisamos informar o tipo da variável.

No JavaScript, não precisamos escrever:

```
String nome  
Number idade
```

Basta escrever:

```
let nome = "Maria";  
let idade = 17;
```

O JavaScript olha para o valor e identifica o tipo.

Por isso dizemos que JavaScript tem **tipagem dinâmica**.

4. `let` e `const`: duas maneiras de criar variáveis

Por enquanto, vamos trabalhar principalmente com `let` e `const`.

Use `let` quando o valor poderá mudar:

```
let pontos = 10;  
  
pontos = 20;  
  
console.log(pontos);
```

Resultado:

```
20
```

Use `const` quando o valor não deve ser substituído:

```
const escola = "Escola Técnica";
```

Depois disso, não devemos fazer:

```
escola = "Outra escola";
```

Existe também `var`, que aparece em materiais antigos de JavaScript:

```
var nome = "Maria";
```

Ela ainda existe, mas em códigos novos vamos preferir `let` e `const`.

5. Três ferramentas para conversar com o usuário

Agora imagine que o JavaScript tem três formas simples de conversar:

alert() é o megafone.

Ele mostra uma mensagem.

```
alert("Bom dia!");
```

prompt() é a pergunta.

Ele pede uma informação ao usuário.

```
let nome = prompt("Qual é o seu nome?");
```

O que a pessoa digitar fica guardado em **nome**.

console.log() é o bloco de anotações dos bastidores.

Ele mostra informações no console do navegador.

```
console.log(nome);
```

No navegador, o resultado pode ser visto nas ferramentas do desenvolvedor, no **Console**.

6. Juntando tudo

Veja este programa:

```
<script>

let nome = prompt("Qual é o seu nome?");
let idade = prompt("Qual é a sua idade?");

alert("Olá, " + nome + "!");
console.log("Nome:", nome);
console.log("Idade:", idade);

</script>
```

A sequência é:

1. Pergunta o nome.
2. Guarda a resposta em uma variável.
3. Pergunta a idade.
4. Guarda a resposta em outra variável.
5. Mostra uma mensagem.
6. Mostra os valores no console.

Essa ideia de **entrada** → **armazenamento** → **saída** aparece o tempo todo em programação.

```
PROMPT
  ↓
variável
  ↓
processamento
  ↓
ALERT / CONSOLE.LOG
```

7. Agora vamos falar de funções

Até aqui, escrevemos cada ação diretamente.

Mas imagine que você criou um procedimento que precisa ser usado várias vezes.

Por exemplo:

Sempre que um jogador entrar no jogo, queremos mostrar uma mensagem de boas-vindas.

Em vez de escrever a mensagem várias vezes, podemos criar uma **função**.

Pense em uma função como uma **máquina**.

Você coloca uma informação nela, a máquina executa uma tarefa e, quando necessário, entrega um resultado.

```
function darBoasVindas() {
    alert("Bem-vindo ao jogo!");
}
```

A função foi criada, mas ainda não foi executada.

Para colocar a máquina para funcionar, chamamos a função:

```
darBoasVindas();
```

Agora o `alert` será exibido.

8. Uma função é como uma receita

Podemos pensar assim:

```
NOME DA RECEITA
    ↓
darBoasVindas()
    ↓
"Mostre uma mensagem de boas-vindas"
```

A vantagem é que podemos usar a mesma função várias vezes:

```
function darBoasVindas() {
    alert("Bem-vindo ao jogo!");
}

darBoasVindas();
darBoasVindas();
darBoasVindas();
```

Não precisamos repetir o código da mensagem.

A função guarda uma tarefa que podemos reutilizar.

9. E se a função precisar de uma informação?

Agora vem uma ideia importante.

Imagine uma máquina de imprimir crachás.

A máquina não pode imprimir o nome de qualquer pessoa. Ela precisa **receber um nome**.

Podemos fazer isso usando um **parâmetro**.

```
function darBoasVindas(nome) {
    alert("Olá, " + nome + "!");
}
```

Aqui:

```
nome
```

é o **parâmetro**.

Ele funciona como um espaço reservado:

```
"Quando alguém usar esta função, coloque alguma informação aqui."
```

Agora podemos chamar a função assim:

```
darBoasVindas("Ana");
```

O resultado será:

```
Olá, Ana!
```

Podemos fazer novamente:

```
darBoasVindas("Carlos");  
darBoasVindas("Julia");  
darBoasVindas("Pedro");
```

A mesma função trabalha com nomes diferentes.

10. Parâmetro e valor enviado

Observe:

```
function darBoasVindas(nome) {  
    alert("Olá, " + nome + "!");  
}
```

nome é o **parâmetro**.

Agora:

```
darBoasVindas("Ana");
```

"Ana" é o valor que estamos enviando para a função.

Podemos imaginar:

```
função  
↓  
darBoasVindas( _____ )  
                ↑  
            espaço para  
            receber um nome
```

Quando fazemos:

```
darBoasVindas("Ana");
```

fica:

```
darBoasVindas( "Ana" )
```

11. Uma função pode receber mais de um parâmetro

Podemos criar uma função para mostrar informações de uma pessoa:

```
function apresentar(nome, idade) {  
    alert("Nome: " + nome + " | Idade: " + idade);  
}
```

Agora podemos usar:

```
apresentar("Ana", 17);
```

Resultado:

```
Nome: Ana | Idade: 17
```

Temos dois parâmetros:

```
nome  
idade
```

E enviamos dois valores:

```
"Ana"  
17
```

A ordem importa:

```
apresentar("Ana", 17);
```

significa:

```
nome ← "Ana"  
idade ← 17
```

12. Funções também podem fazer contas

Uma função não precisa apenas mostrar mensagens.

Ela também pode realizar uma tarefa e devolver um resultado.

Por exemplo:

```
function soma(a, b) {  
  return a + b;  
}
```

Aqui:

- `a` é o primeiro parâmetro.
- `b` é o segundo parâmetro.
- `return` devolve o resultado.

Podemos usar assim:

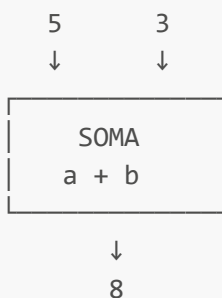
```
let resultado = soma(5, 3);
```

```
console.log(resultado);
```

Resultado:

8

Pense na função como uma máquina:



A mesma máquina pode receber outros números:

```
console.log(soma(10, 2));  
console.log(soma(7, 8));  
console.log(soma(100, 50));
```

13. Um pequeno programa completo

Agora vamos juntar tudo.

```
<!DOCTYPE html>  
<html>  
  <head>  
    <title>Jogo</title>  
  </head>  
  <body>  
  
    <script>  
  
    function apresentarJogador(nome, pontos) {  
      alert("Jogador: " + nome);  
      console.log("Pontos:", pontos);  
    }  
  
    let nome = prompt("Qual é o nome do jogador?");  
    let pontos = 10;
```

```
apresentarJogador(nome, pontos);

</script>

</body>
</html>
```

Observe a sequência:

```
prompt()
  ↓
guarda a resposta em uma variável
  ↓
chama uma função
  ↓
a função recebe os parâmetros
  ↓
alert() e console.log()
```

É isso que estamos construindo: pequenas peças de código que recebem informações e realizam tarefas.

Atividade — faça no caderno

Não é necessário computador. Leia o código e responda.

1. Descobrindo o que está nas caixas

Observe:

```
let nome = "Lucas";
let idade = 16;
let aprovado = true;
```

Escreva:

```
nome → _____
idade → _____
aprovado → _____
```

Agora escreva o **tipo** de cada variável.

2. O que aparecerá na tela?

```
let nome = "Marina";  
  
alert("Olá, " + nome);
```

Qual será a mensagem exibida?

3. Complete o código

A função abaixo deve receber o nome de uma pessoa e mostrar uma mensagem.

```
function cumprimentar(_____) {  
    alert("Olá, " + _____ + "!");  
}  
  
cumprimentar("João");
```

Qual palavra deve aparecer nos dois espaços?

4. Descubra os parâmetros

Observe:

```
function mostrarAluno(nome, turma) {  
    console.log("Aluno:", nome);  
    console.log("Turma:", turma);  
}
```

Quais são os parâmetros da função?

5. Descubra o resultado

```
function multiplicar(a, b) {  
    return a * b;  
}  
  
let resultado = multiplicar(4, 3);  
  
console.log(resultado);
```

O que aparecerá no console?

6. Monte a sequência

Observe:

```
let nome = prompt("Qual é o seu nome?");

function mensagem(nome) {
    alert("Olá, " + nome + "!");
}

mensagem(nome);
```

No caderno, escreva o que acontece em cada etapa:

1. _____
2. _____
3. _____
4. _____

Desafio

Crie no caderno uma função chamada `mostrarPersonagem`.

Ela deve receber:

```
nome
poder
```

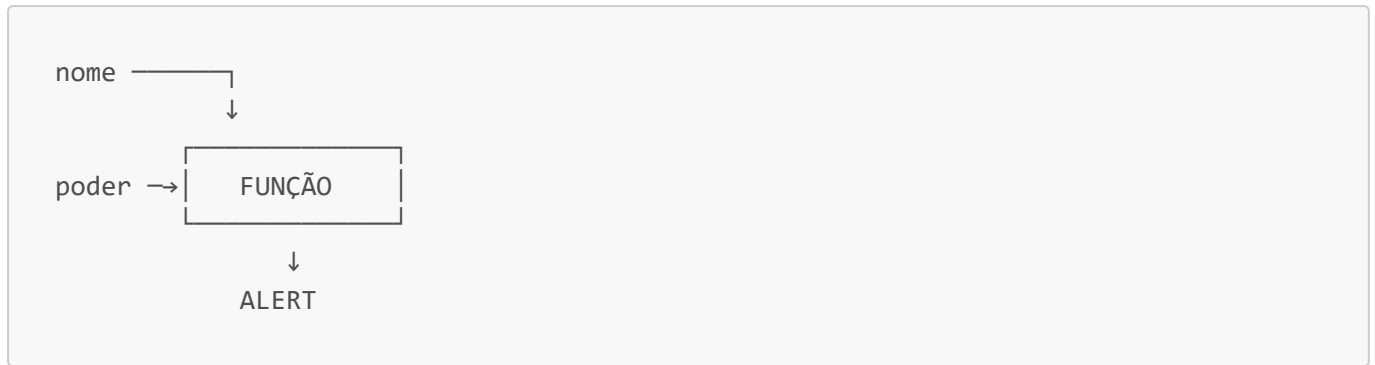
E mostrar um `alert` com uma mensagem parecida com:

```
Personagem: Mario
Poder: Super salto
```

Depois escreva uma chamada da função usando os valores:

```
Mario
Super salto
```

Tente desenhar a função como uma máquina:



Para guardar

Uma variável é como uma **caixa com nome**.

O tipo diz **o que está dentro da caixa**.

Uma função é como uma **máquina que executa uma tarefa**.

Um parâmetro é um **espaço que a função deixa preparado para receber uma informação**.

E o JavaScript pode conversar com o usuário usando:

```
prompt()
alert()
console.log()
```